

Predicados de segundo orden y extralógicos

Curso Programación Lógica 2024

Predicados de segundo orden

- `findall/3`

`findall(Object, Goal, List)`: devuelve en `List` todos los objetos que satisfacen `Goal`

Predicados de segundo orden

- `findall/3`

`findall(Object, Goal, List)`: devuelve en `List` todos los objetos que satisfacen `Goal`

"Obtener todos los pares de enteros que suman 10"

Predicados de segundo orden

- `findall/3`

`findall(Object, Goal, List)`: devuelve en `List` todos los objetos que satisfacen `Goal`

"Obtener todos los pares de enteros que suman 10"

?- `findall(par(X,Y),sumaint(X,Y,10),L).`

Predicados de segundo orden

- bagof/3

bagof(Object,Goal,List): devuelve en List todos los objetos que satisfacen Goal. Si una variable está en Goal pero no en Object, itera sobre todos los valores posibles.

Diferencia con findall: si no hay soluciones, falla.

- setof/3: ordena los resultados y elimina duplicados

Predicados de segundo orden

`edad(7,luis).`

`edad(6,pedro).`

`edad(6,alejandra).`

`edad(6,alejandra).`

`edad(5,ana).`

`edad(5,luisa).`

`:- bagof(Nombre,edad(Edad,Nombre),L).`

`:- setof(Nombre,edad(Edad,Nombre),L).`

Ejercicio 9 [prueba 15]

Considere un laberinto modelado mediante una matriz de enteros en Prolog. Las celdas de la matriz pueden valer 0 cuando se permite pasar o 1 cuando no se permite (es una pared). Por ejemplo, la siguiente matriz representa un laberinto:

```
0 1 0 0 0 0
0 1 0 1 1 0
0 0 0 0 0 0
0 1 0 1 0 1
```

La representación interna de la matriz no es conocida, se cuenta solamente con el siguiente predicado para su manejo:

`celda(+L, +I, +J, ?V)` `<- V` es el valor de la celda en la fila I y columna J de la matriz L.

Si la fila o la columna no existen, el predicado falla.

a) `adyacente_valido(+L,+I1,+J1,?I2,?J2) <-` $(I1, J1)$ e $(I2, J2)$ son celdas adyacentes en L y además se puede pasar de una a la otra (las dos valen 0). Los movimientos válidos son horizontal y verticalmente, no hay movimiento en diagonal.

b) `camino(+L, +I1, +I2, ?J1, ?J2, ?C) <-` C es un camino para ir de la celda $(I1, J1)$ a la celda $(I2, J2)$ de L. C es una lista de elementos de la forma `celda(X,Y)`.

c) `camino_mas_corto(+L, +I1, +I2, ?J1, ?J2, ?C) <-` C es el camino más corto (o uno de los caminos más cortos si hay varios de la misma distancia) para ir de la celda $(I1, J1)$ a la celda $(I2, J2)$ de L.

d) `alcanzables(+L,+I,+J,+N,?C) <-` C es una lista con todas las celdas de L para las que existe un camino desde la celda (I, J) que pase como máximo por N celdas. La lista C puede tener elementos repetidos.

Ejercicio 9

% Camino en un grafo

```
conectado(X,Y) :- conectado(X,Y,[X]).
```

```
conectado(X,X,_).
```

```
conectado(X,Y,Visitados):-
```

```
    arista(X,N),
```

```
    \+ member(N,Visitados),
```

```
    conectado(N,Y,[N|Visitados]).
```

Ejercicio 9

`adyacente_valido(L,I1,J1,I2,J2):- ...`

`camino(L,I1,J1,I2,J2,C):- ...`

`camino_mas_corto(L,I1,J1,I2,J2,X):-`

Ejercicio 9

```
adyacente_valido(L,I1,J1,I2,J2):- ...
```

```
camino(L,I1,J1,I2,J2,C):- ...
```

```
camino_mas_corto(L,I1,J1,I2,J2,X):-  
    findall(C,camino(L,I1,J1,I2,J2,C),Caminos),  
    mas_corto(Caminos,X).
```

Ejercicio 9

`alcanzable(L,I1,J1,N,Celdas):-`

Ejercicio 9

```
alcanzable(L,I1,J1,N,Celdas):-  
    findall(celda(X,Y),alcanzable_N(L,I1,J1,X,Y,N),Celdas).
```

```
alcanzable_N(L,I1,J1,I2,J2,N):-  
    camino(L,I1,J1,I2,J2,C),  
    length(C,M),M1 is M-1,  
    M1 =<N.
```

Predicados extralógicos

- Predicados con efectos colaterales
- Entrada/salida
- Ciclos loop/fail
- Modificación de la base de programas

Ciclos loop-fail

- Ciclos loop-fail: queremos revisar todos los resultados en una sola invocación.
- Tiene que ser extralógico
- Mostrar todos los pares que suman T

% Mostrar todos los pares que suman T.

```
mostrar_pares(T):-  
    sumaint(X,Y,T),...
```

-

Ciclos loop-fail

- Ciclos loop-fail: queremos revisar todos los resultados en una sola invocación.
- Tiene que ser extralógico
- Mostrar todos los pares que suman T

% Mostrar todos los pares que suman T.

```
mostrar_pares(T):-  
    sumaint(X,Y,T),  
    writeln(par(X,Y)),  
    fail.  
mostrar_pares(_).
```


Manipulación de programas

- Acceder a la base de programas... y alterarla.

```
clause(+Head,?Body)
```

```
?- clause(largo(X,Y),Z).
```

```
Z = (nonvar(X), largo1(X, Y)) ;
```

```
Z = (var(X), nonvar(Y), largo2(X, Y)).
```

Manipulación de programas

- Acceder a la base de programas... y alterarla.

Podemos agregar y quitar hechos y cláusulas...

`assert(+Clause)`

`assertz(+Clause)` - Agrego una cláusula al final del mi programa

`asserta(+Clause)` - Agrego una cláusula al comienzo de mi programa

`retract(+Clause)` - Quito la primera cláusula que unifique con Clause

`retractall(+Head)` - Quito todas las cláusulas cuyo lado izquierdo unifique con Head

Manipulación de programas

- Fibonacci

```
fib(0, 1) :- !.  
fib(1, 1) :- !.  
fib(N, Fib) :-  
    N > 1,  
    N1 is N-1,  
    N2 is N-2,  
    fib(N1, F1),  
    fib(N2, F2),  
    Fib is F1+F2.
```

Manipulación de programas

Fibonacci

```
fib(0, 1) :- !.
```

```
fib(1, 1) :- !.
```

```
fib(N, Fib) :-
```

```
    N > 1,
```

```
    N1 is N-1,
```

```
    N2 is N-2,
```

```
    fib(N1, F1),
```

```
    fib(N2, F2),
```

```
    Fib is F1+F2.
```

```
?- time(fib(30,F)).
```

```
% 5,385,073 inferences, 0.391 CPU in 0.414 seconds
```

```
(94% CPU, 13785787 Lips)
```

```
F = 1346269.
```

Manipulación de programas

Fibonacci (dinámico!)

```
fib1(0, 1) :- !.
```

```
fib1(1, 1) :- !.
```

```
fib1(N, Fib) :-
```

```
    N > 1,
```

```
    N1 is N-1,
```

```
    N2 is N-2,
```

```
    fib1(N1, F1),
```

```
    fib1(N2, F2),
```

```
    Fib is F1+F2.
```

```
?- time(fib1(30,F)).
```

```
% 146 inferences, 0.000 CPU in 0.000 seconds (0%  
CPU, Infinite Lips)
```

```
F = 1346269.
```

Manipulación de programas

Fibonacci (dinámico!)

`:- dynamic fib1/2`

```
fib1(0, 1) :- !.
```

```
fib1(1, 1) :- !.
```

```
fib1(N, Fib) :-
```

```
    N > 1,
```

```
    N1 is N-1,
```

```
    N2 is N-2,
```

```
    fib1(N1, F1),
```

```
    fib1(N2, F2),
```

```
    Fib is F1+F2,
```

```
    asserta((fib1(N,Fib) :- !)).
```

```
?- time(fib1(30,F)).
```

```
% 146 inferences, 0.000 CPU in 0.000 seconds (0%
```

```
CPU, Infinite Lips)
```

```
F = 1346269.
```



setarg y nb_setarg

- Ejercicio 4: implementar matriz como funtores

`m(f(1,2,3),f(4,5,6))`

- `nuevo_valor(+M,+I,+J,+V)`: asigna el valor `V` a la celda `(i,j)`, *modificando el término*

`nuevo_valor(M,I,J,V):-`

`M =.. [_|Args],`

`nth1(I,Args,Fila), % nth1 devuelve el i-ésimo elemento`

`setarg(J,Fila,V).`

setarg

- Ejercicio crear una matriz para reflejar cuáles pares suman T

?- resultados_suma(5,MatrizResultados).

```
MatrizResultados = m(f(o, o, o, x, o), f(o, o, x, o, o), f(o, x,  
o, o, o), f(x, o, o, o, o), f(o, o, o, o, o))
```


setarg

- Ejercicio crear una matriz para reflejar cuáles pares suman T

```
resultados_suma(T,MatrizResultados):-  
    matriz(T,T,o,MatrizResultados),  
    resultados_suma1(T,MatrizResultados).
```

```
resultados_suma1(T,MatrizResultados):-  
    sumaint(X,Y,T),  
    nuevo_valor(MatrizResultados,X,Y,x),  
    fail.  
resultados_suma1(_,_).
```

setarg

- Ejercicio crear una matriz para reflejar cuáles pares suman T

```
?- resultados_suma(5,MatrizResultados).
```

```
MatrizResultados = m(f(o, o, o, o, o), f(o, o, o, o, o), f(o, o,  
o, o, o), f(o, o, o, o, o), f(o, o, o, o, o))
```

¿Qué pasó?

nb_set_arg

- nuevo_valor(+M,+I,+J,+V): asigna el valor V a la celda (i,j), *modificando el término, con un efecto que no se revierte al hacer backtracking*

```
nuevo_valor(M,I,J,V):-  
    M =.. [_|Args],  
    nth1(I,Args,Fila), % nth1 devuelve el i-ésimo elemento  
    nb_setarg(J,Fila,V).
```

setarg y nb_setarg

- Ejercicio crear una matriz para reflejar cuáles pares suman T

```
?- resultados_suma(5,MatrizResultados).
```

```
MatrizResultados = m(f(o, o, o, x, o), f(o, o, x, o, o), f(o, x,  
o, o, o), f(x, o, o, o, o), f(o, o, o, o, o))
```

Referencias

- The Art of Prolog. Capítulo 10
- The Art of Prolog. Capítulo 12
- Documentación de SWI Prolog